

High-Level Synthesis with Bambu on a Reproducible Workbench

C behaviour to verified Verilog RTL, end-to-end through the Crucible CLI

A six-line C function, synthesized to register-transfer-level (RTL) Verilog with Panda Bambu and then proven to still compute what the C said — every step driven by igny, in one version-locked workspace.

#	STAGE	TOOL (VIA IGNY)	RESULT
1	C → Verilog RTL synthesis	bambu 2024.10	✓ PASS
2	C-vs-RTL equivalence check	iverilog 14.0.0, vvp	✓ PASS

Two stages, both passing, both reproducible from a committed `crucible.lock` on any machine.

CONTENTS

Contents

01	Executive Summary	3
02	Why HLS, and Why Bambu	4
03	The Reference Kernel	4
04	Reproducible Workspace Setup	5
05	Experiment — Synthesis to RTL	6
06	Experiment — Equivalence Verification	7
07	What the Generated RTL Reveals	8
08	Conclusion and Source Notes	9

How to read this note

Every command in this document was run as printed, and every figure is captured output — no reconstructed logs. The synthesis and verification numbers in §05–§07 come from a single run of `scripts/run_hls.sh` in the workspace described in §04.

SECTION 01

Executive Summary

High-level synthesis promises to compile ordinary C into hardware. The promise is easy to state and hard to trust: the moment a compiler is free to reorder, share, and re-time the operations you wrote, you need evidence that the circuit it emitted still means what your program meant.

This lab note runs that loop end-to-end on a deliberately small kernel. Using Panda Bambu through the Crucible CLI (`igny`), we synthesize a six-line C function into Verilog RTL, then gate the result on a C-vs-RTL equivalence check driven by Icarus Verilog. Both stages pass, and both are reproducible on any machine from a committed lock file.

The interesting result is not that it worked. It is what the generated datapath looks like: Bambu did not instantiate an adder and a subtractor and multiplex their results. It negated one operand, multiplexed the **operand**, and shared a single adder across both arms of the branch — the resource-sharing transformation that makes HLS worth using, made visible on a kernel small enough to check by hand.

WHAT THIS NOTE ESTABLISHES

CLAIM	EVIDENCE	STATUS
C compiles to synthesizable RTL	<code>myMacro.v</code> — 1 FSM state, 0 registers	✓ SHOWN
RTL preserves C behaviour	6 / 6 vectors match golden	✓ SHOWN
Bambu shares operators	one adder serves both branches	✓ SHOWN
The flow is reproducible	<code>crucible.lock</code> + <code>workspace</code> sync	✓ SHOWN

One caveat worth stating early

The C source returns a `long`. The synthesized interface returns 32 bits, not 64. Bambu sized the port to the width the computation actually needs — correct for these operands, and exactly the kind of specification-versus-interface gap an equivalence check exists to surface. See §07.

SECTION 02

Why HLS, and Why Bambu

Register-transfer-level design asks an engineer to describe hardware cycle by cycle. High-level synthesis (HLS) raises that starting point: the engineer writes the algorithm in C or C++, and the tool schedules the operations into cycles, allocates the functional units, and binds values to registers. The design space it explores — how many adders, how many cycles, what runs in parallel — is the part a human does slowly and a compiler does exhaustively.

Bambu is the open-source HLS compiler from the PandA group at Politecnico di Milano. It matters here for two reasons: it is genuinely open, so every intermediate result in this note can be inspected rather than taken on faith; and it is already catalogued in the Ignytion tool registry, so it installs and version-locks like any other Crucible tool.

SECTION 03

The Reference Kernel

One function, chosen to be trivially checkable by hand while still containing the one thing that makes HLS non-trivial: a data-dependent branch over two different arithmetic operations.

SOURCE — SRC/MY_MACRO.C

```
// c = 1 -> j + k    c = 0 -> j - k
long myMacro(int j, int k, bool c)
{
    long i = 0;
    if (c) {
        i = j + k;
    } else {
        i = j - k;
    }
    return i;
}
```

BEHAVIOURAL SPECIFICATION

INPUT C	OUTPUT	INTERPRETATION
0	j - k	Subtract path
1	j + k	Add path

Two operands, one control bit, one result. The whole truth table is two rows — which is the point: every claim made about the generated hardware in this note can be checked against it directly.

SECTION 04

Reproducible Workspace Setup

The failure mode this addresses is mundane and universal: an HLS result that reproduces only on the machine that produced it. Crucible makes the toolchain part of the repository. One environment, one workspace, two tools, one lock file.

PROCEDURE — ONE-TIME SETUP

```
# activate the Crucible sandbox, then create an isolated toolbox
igny env create work-hls
igny workspace create --path . --env work-hls

# install the HLS compiler and the reference simulator
igny tool install bambu
igny tool install iverilog

# freeze the exact versions into crucible.lock
igny workspace lock
```

LOCKED TOOLCHAIN

TOOL	VERSION	ROLE
bambu	2024.10	PandA HLS compiler — C to Verilog RTL
iverilog	14.0.0	Reference simulator for the equivalence gate

A collaborator reproduces the entire environment with `git clone` followed by `igny workspace sync` — the lock file names the exact artefacts, so the toolchain that produced the numbers on the following pages is the toolchain they get.

Why the equivalence check uses Icarus, not Bambu's own co-simulation

Bambu ships a native `--simulate` co-simulation mode. It compiles a C++ testbench harness, so it needs a host C/C++ toolchain present at run time. The hand-written Verilog testbench used here needs only the simulator that is already version-locked in the workspace — fewer moving parts, and the check stays reproducible wherever the lock file goes.

SECTION 05 · EXPERIMENT

Synthesis — C Behaviour to Verilog RTL

AIM

Compile `src/c_source_code.c` into synthesizable Verilog with the C function as the top-level entity, and read the scheduling and allocation report Bambu emits.

PROCEDURE

```
igny run bambu src/c_source_code.c --top-fname=myMacro \  
    --output-temporary-directory=results/my_macro/panda-temp
```

SYNTHESIS REPORT — CAPTURED OUTPUT

```
Analyzing function myMacro  
Parameter j   Bitwidth : 32      Parameter k   Bitwidth : 32  
Parameter c   Bitwidth :  1  
  
Number of control steps: 3      Minimum slack: 5.615  
Estimated max frequency (MHz): 228.05  
Number of states: 1            Minimum number of cycles: 1  
Bound operations: 7/7          Number of modules instantiated: 7  
Total estimated area: 124      Estimated number of DSPs: 0  
Register allocation obtains an optimal result: 0 registers  
Total number of flip-flops in function myMacro: 0
```

GENERATED TOP-LEVEL INTERFACE

PORT	WIDTH	DIRECTION AND ROLE
clock, reset	1	In — reset is active low
start_port	1	In — one-cycle pulse begins execution
j, k	32	In — the two integer operands
c	1	In — the control bit
done_port	1	Out — asserted when the result is valid
return_port	32	Out — the computed result

Zero flip-flops and a single state: Bambu recognised the function as purely combinational and wrapped it in the standard handshake rather than pipelining it. The result is available in the cycle after `start_port` is asserted.

SECTION 06 · EXPERIMENT (CONTD.)

Equivalence — Does the RTL Still Mean the C?

AIM

Drive the synthesized module through its handshake with vectors covering both branches, and compare every result against the golden value from the C specification. A single mismatch fails the run.

PROCEDURE

```
igny run iverilog -o results/my_macro/my_macro_sim -s verilog_code_tb \
    results/my_macro/myMacro.v src/verilog_code_tb.v
igny run vvp results/my_macro/my_macro_sim
```

RESULT — CAPTURED OUTPUT

```
C-vs-RTL equivalence check: myMacro (Bambu RTL vs C reference)
PASS myMacro(1, 2, 0) -> RTL=-1 golden=-1
PASS myMacro(1, 2, 1) -> RTL=3 golden=3
PASS myMacro(10, 3, 0) -> RTL=7 golden=7
PASS myMacro(10, 3, 1) -> RTL=13 golden=13
PASS myMacro(0, 0, 0) -> RTL=0 golden=0
PASS myMacro(-5, 7, 1) -> RTL=2 golden=2

==== EQUIVALENT: Bambu RTL matches the C reference on all vectors ====
// >> GATE PASSED: RTL is equivalent to the C reference on all vectors
```

INTERPRETATION

The vector set covers both arms of the branch, a negative result, a negative operand, and the zero case. Signed values survive the round trip intact — `myMacro(1, 2, 0)` returns `-1` from a 32-bit port, confirming the two's-complement interpretation matches C's.



Experiment — PASS

6 / 6 vectors match the C reference. Gate passed; `equiv.log` retained as evidence in `results/my_macro/`.

SECTION 07

What the Generated RTL Reveals

A passing equivalence check says the hardware is correct. Reading the datapath says what the compiler actually did — and this is where a kernel this small earns its keep.

FINDING 01 — THE MUX MOVED, AND THE ADDER IS SHARED

The obvious implementation of $c \ ? \ j+k \ : \ j-k$ instantiates an adder and a subtractor and multiplexes their outputs. Bambu did something better. It negates k , multiplexes between k and $-k$ — selecting the **operand**, not the result — and feeds one adder:

```
// datapath_myMacro - instantiation order, from the generated RTL
ui_negate_expr_FU      negate      : -k
ui_extract_bit_expr_FU select      : bit 0 of c
ui_cond_expr_FU        operand mux : select ? k : -k
ui_plus_expr_FU        adder       : j + (selected operand)

assign return_port = out_ui_plus_expr_FU_32_32_32_6_i0_...;
```

One adder, one negator, one 32-bit mux — against two adders and one mux for the naive version. This is operator sharing: the transformation that makes HLS produce smaller hardware than a literal transcription of the source, and the reason the output cannot simply be assumed to mirror the input structure.

FINDING 02 — THE RETURN TYPE NARROWED

The C function returns `long` — 64 bits on the host. The synthesized `return_port` is 32 bits. Bambu sized the interface to the width the computation demonstrably needs, since both operands are 32-bit `int`. For these vectors that is correct and provably so; for a kernel where the accumulated result can exceed 32 bits, it is a specification gap that only an equivalence check with the right vectors would expose.

The general lesson

Both findings point the same way. HLS output is not a transcription of the source — it is the result of scheduling, allocation, and binding decisions the compiler made on your behalf. That is precisely why the equivalence gate belongs in the loop rather than at the end of it.

SECTION 08

Conclusion

A six-line C function became verified Verilog RTL in two commands, inside an environment that a collaborator can reconstruct exactly from a lock file. The synthesis pass produced a combinational datapath at an estimated 228 MHz with zero registers; the verification pass matched all six vectors against the C reference.

The substantive result is the datapath itself. Bambu shared a single adder across both arms of the branch by moving the multiplexer to the operand side, and narrowed a `long` return to the 32 bits the computation needs. Neither decision is visible in the source, and neither is wrong — but both are exactly the kind of transformation that makes "the C compiles" and "the hardware is correct" two different claims requiring two different kinds of evidence.

That gap is where this work continues. The check demonstrated here is vector-based: it proves agreement on the cases tested. Formal equivalence over all inputs — modelling both the C and the RTL as finite state machines with datapath and proving the synthesis transform preserved meaning — is the natural next gate, and the point at which an agent could be trusted to iterate on an HLS design without a human reading every netlist.

REPRODUCE THIS DOCUMENT

```
git clone <repo> && cd HLS_experiment
igny workspace sync
bash scripts/run_hls.sh
# HLS synthesis, then the C-vs-RTL equivalence gate
```

SOURCE NOTES

PandA Bambu — open-source HLS compiler, Politecnico di Milano. Version 2024.10 used throughout.
panda.dei.polimi.it

Icarus Verilog — event-driven Verilog simulator. Version 14.0.0, installed via the Crucible catalogue.
steveicarus.github.io/iverilog

Crucible CLI (igny) — environment orchestration and tool version locking. Community tier.
ignytion.io

Experiment workspace — sources, testbench, runner script, and captured results.
[HLS_experiment/](#) · run of September 2026